

Relatório sobre o 3º trabalho prático de EDA 2

Card Exchange



Universidade: Colégio Luís António Verney (Universidade de Évora)

Curso: Engenharia Informática

Identificações: Grupo g302

(Diogo Tavares Nº 58049 ,Francisco Rodrigues Nº 59119)

Ano: 2024/25

Objetivo do Trabalho

O objetivo deste trabalho é determinar se num festival de trocas de cartas cada participante pode sair do festival com uma carta que este cobiça, após fazer as trocas necessárias.

Sendo que cada participante entra na festa com a sua própria carta e declara que quer uma carta diferente de outro participante.

Assim temos 2 entidades neste esquema: As cartas e os participantes.

Também sabemos que a única relação estabelecida entre as duas entidades é a de que cada carta pode ou não ser cobiçada por outro participante.

Assim podemos diretamente representar as entidades e as suas preferências através de uma rede de fluxos.

Características da rede

A nossa interpretação da rede de fluxos presente no problema é a de que existem 4 tipos de nós:

- Fonte**(origem de todo o fluxo presente na rede)
- Cartas**(representando as cartas de cada participante)
- Participantes**(representando os participantes)
- Dreno**(onde acaba todo o fluxo que passa pela rede)

Através destas 4 entidades estabelecemos ainda ligações entre os nós da seguinte forma:

- a fonte estabelece ligações para todas as cartas com capacidade 1
- Cada carta tem uma ligação, de capacidade 1, para cada participante que a cobiça
- Cada participante tem uma ligação ao dreno , também de capacidade 1.

Portanto podemos realizar o objetivo do trabalho através desta rede de fluxos pois sabemos que no caso em que o fluxo que entra na rede pela fonte é o mesmo que sai da rede pelo dreno então quer dizer que podem ser feitas trocas de maneira a que todos os participantes saiam do festival com uma carta que cobiçam.

Caso contrário não é possível que todos os participantes saiam do festival com a carta que cobiçam

Implementação da rede

Para implementar a rede de fluxos, utilizamos principalmente duas estruturas de dados: listas de adjacência(adj) e arestas(FlowEdge).

A rede é representada como um vetor de listas ($\text{List}<\text{FlowEdge}>[]$), onde cada posição do vetor corresponde a um nó da rede (seja ele a fonte, uma carta, um participante ou o dreno), e cada lista contém todas as arestas associadas a esse nó.

Descrição do Algoritmo

Introdução:

Primeiramente precisamos entender que para verificar o objetivo do trabalho através da existente rede de fluxos podemos verificar que se o fluxo máximo da rede for igual ao número de participantes(N) então o objetivo é concluído e todos os participantes obtêm 1 carta cobiçada.

Para realizar esta verificação e achar o fluxo máximo utilizamos o algoritmo de Edmonds–Karp, uma implementação clássica do método de Ford–Fulkerson em conjunto com a BFS para cálculo de fluxo máximo em redes.

Ele consiste em encontrar, de forma iterativa, caminhos aumentantes da fonte ao dreno, utilizando busca em largura (BFS) no grafo residual.

Etapas do algoritmo:

- Inicialização:
 - Começa com fluxo total igual a zero.
 - Cria um vetor `edgeTo[]` para cada nó, para armazenar a aresta usada para chegar até ele durante a BFS.
- Busca por Caminho Aumentante (BFS):
 - Em cada iteração, realiza-se uma BFS a partir da fonte.
 - Durante essa busca, apenas se considera seguir por arestas com capacidade residual positiva.
 - Se a BFS encontrar um caminho até o dreno, este caminho é chamado de caminho aumentante.
- Atualização de Fluxo:
 - Após encontrar um caminho aumentante, determina-se o bottleneck (gargalo), que é a menor capacidade residual ao longo do caminho.
 - Esse valor é então adicionado ao fluxo de todas as arestas no caminho.
 - Também são atualizadas as arestas reversas (do grafo residual), que permitem desfazer o fluxo se necessário.
- Repetição:
 - O processo de busca e atualização repete-se enquanto houver caminhos aumentantes da fonte ao dreno.

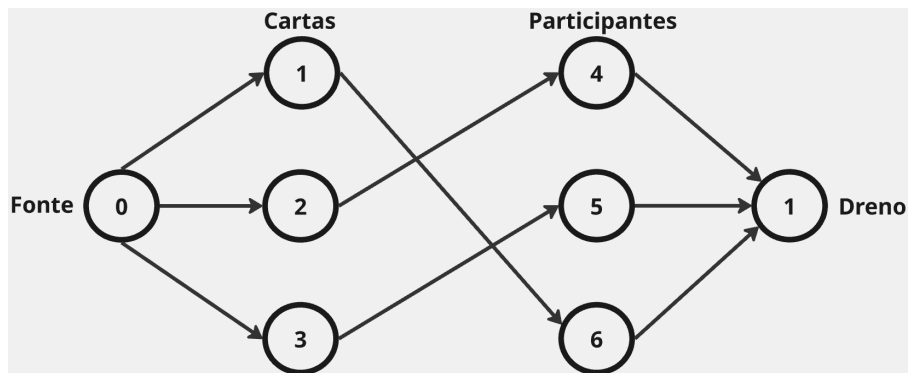
- Finalização:
 - Quando não for mais possível encontrar caminhos aumentantes, o algoritmo termina.
 - O valor do fluxo máximo corresponde à soma dos fluxos enviados ao longo desses caminhos.

Exemplo do Algoritmo

Exemplo de Input:

3 3
0 1
1 2
2 0

Representação da Rede:



Contexto:

$N = 3$ (3 participantes, cada um quer uma carta em específico)

$M = 3$ (3 declarações de preferência).

Declarações:

Participante 0 quer a carta do participante 1 (carta 2).

Participante 1 quer a carta do participante 2 (carta 3).

Participante 2 quer a carta do participante 0 (carta 1).

Funcionamento do Código:

1. Estrutura da Rede

- Fonte (0), Dreno (7)
- Cartas: Nós 1, 2, 3 (cada carta pertence a cada um)
- Participantes: Nós 4, 5, 6 (cada participante quer receber uma carta).

2. Arestas:

- Fonte $\rightarrow$ Cartas: $0 \rightarrow 1$, $0 \rightarrow 2$, $0 \rightarrow 3$ (capacidade 1, cada carta deve ser distribuída 1 única vez).
- Participantes $\rightarrow$ Dreno: $4 \rightarrow 7$, $5 \rightarrow 7$, $6 \rightarrow 7$ (capacidade 1, cada participante deve receber exatamente uma carta).

3. Conexões baseadas nas preferências:

- Participante 0 (nó 4) quer a carta 2 (do participante 1):
 $2 \rightarrow 4$ (carta 2 $\rightarrow$ participante 0)
- Participante 1 (nó 5) quer a carta 3 (do participante 2):
 $3 \rightarrow 5$ (carta 3 $\rightarrow$ participante 1)
- Participante 2 (nó 6) quer a carta 1 (do participante 0):
 $1 \rightarrow 6$ (carta 1 $\rightarrow$ participante 2)

4. Fluxo Máximo:

O algoritmo encontra 3 caminhos aumentantes:

$0 \rightarrow 1 \rightarrow 6 \rightarrow 7$ (fluxo 1)

$0 \rightarrow 2 \rightarrow 4 \rightarrow 7$ (fluxo 1)

$0 \rightarrow 3 \rightarrow 5 \rightarrow 7$ (fluxo 1)

Fluxo total = 3 (igual a N).

5. Output:

“YES”

Explicação:

O código verifica se todas as cartas podem ser redistribuídas conforme as preferências. Neste caso, há um ciclo perfeito ($0 \rightarrow 1 \rightarrow 2 \rightarrow 0$), permitindo que todos recebam a carta desejada. O fluxo máximo igual a N confirma que é possível.

Complexidade Temporal

N = Número de participantes.

M = Número de declarações de preferência.

Explicação:

1. O algoritmo **Edmonds-Karp** tem complexidade $O(V * E^2)$ no pior caso:

V = número de nós na rede de fluxo.

E = número de arestas.

2. No código:

$V = 2N + 2$ (fonte + dreno + N nós para cartas + N nós para participantes).

$E = 2N + M$ (N arestas fonte→cartas + N arestas participantes→dreno + M arestas declarações de preferências).

3. Substituindo na fórmula:

$O((2N + 2) * (2N + M)^2)$, ou seja, a complexidade temporal do código é
 $O(N * (N + M)^2)$

Complexidade Espacial

Explicação:

1. Rede de fluxo (Flow Network):

Nós: A rede tem $2N + 2$ nós (fonte, dreno, N cartas, N participantes).

Arestas:

$2N$ arestas (fonte $\rightarrow$ cartas + participantes $\rightarrow$ dreno).

M arestas (preferências entre cartas e participantes).

Total de arestas: $E = 2N + M$.

Espaço para listas de adjacência: $O(E)$ (cada aresta é armazenada duas vezes).

2. Estruturas auxiliares:

-edgeTo[], visited[]: Arrays de tamanho $O(V) = O(N)$

-Fila para BFS: Armazena até $O(V)$ nós.

3. Conclusão:

Dominada pela rede de fluxo: $O(V + E)$, ou seja, a complexidade espacial do código é **$O(N + M)$**